Week 06

파일 · CSV · 예외 처리

AI활용코딩연습(ZBE135)
이영호 교수 · 국립목포대학교 컴퓨터학부

지난주 돌아보기

```python
def letter_grade(total):
    if not 0 <= total <= 100:
        raise ValueError(f"총점은 0~100이어야 합니다: {total}")
    ...

def test_letter_grade():
    assert letter_grade(90) == "A"
```

- 계산 함수는 `return`, 잘못된 입력은 `raise`
- 오늘은 `raise`된 오류를 **받아서 처리**하는 법

오늘의 목표

1. `with open`과 `pathlib`으로 파일을 안전하게 쓰고 읽는다
2. `csv.DictReader` / `DictWriter`로 CSV를 다룬다 — **값은 모두 문자열**
3. `try / except / else / finally`로 예상 가능한 오류를 처리한다
4. 오류가 섞인 성적 파일을 **정상 / 오류 행으로 나누고 보고서**를 만든다
5. `except: pass`가 **버그를 숨기는** 위험을 확인한다

수업 흐름

구간	시간	활동
도입 · 개념	20분	파일 모드, CSV, 예외 처리 구조
안내 실습	45분	텍스트 파일, CSV 읽고 쓰기, try/except/raise
독립 · 짝 실습	35분	오류에 견고한 성적 파일 처리기
공유 · 점검	20분	`except: pass` 사례, 과제 안내

이번 주 Colab 실습 열기

실제 데이터는 지저분하다

```
student_id,name,department,year,midterm,final,assignment,attendance
20261011,김민준,컴퓨터공학과,2,78,85,90,15
20261014,,경영학과,2,80,75,82,16          ← 이름 없음
20261015,정도윤,컴퓨터공학과,4,abc,77,88,13  ← 숫자가 아님
20261016,강지우,간호학과,3,88,120,91,16     ← 범위 밖
20261012,이서연,정보보안학과,3,92,88,95,16  ← 중복
```

`students_raw.csv` 18행 중 **8행에 문제** — 프로그램이 멈추면 안 된다

파일 열기 모드

모드	의미	파일이 없으면	파일이 있으면
`"r"`	읽기	`FileNotFoundError`	처음부터 읽기
`"w"`	쓰기	새로 만듦	**내용 삭제** 후 쓰기
`"a"`	이어 쓰기	새로 만듦	끝에 덧붙임

```
with open("memo.txt", "w", encoding="utf-8") as f:
    f.write("1주차: Colab 시작\n")
```

- `with` → 블록이 끝나면 **자동으로 닫힘**
- 한글은 **`encoding="utf-8"`** 필수

한 줄씩 읽기와 pathlib

```
with open("memo.txt", encoding="utf-8") as f:
    for line_no, line in enumerate(f, 1):
        print(line_no, line.strip())      # 줄 끝 '\n' 제거
```

```
from pathlib import Path
path = Path("memo.txt")
path.exists()                             # True
text = path.read_text(encoding="utf-8")
Path("out").mkdir(exist_ok=True)
(Path("out") / "summary.txt").write_text("...", encoding="utf-8")
```

csv.DictReader

```
import csv
with open("students_raw.csv", encoding="utf-8", newline="") as f:
    rows = list(csv.DictReader(f))

rows[0]
# {'student_id': '20261011', 'name': '김민준', ..., 'midterm': '78', ...}

rows[0]["midterm"] + rows[0]["final"]            # '7885'  ← 이어 붙임!
int(rows[0]["midterm"]) + int(rows[0]["final"])  # 163
```

> CSV 값은 **모두 문자열**, 빈 칸은 `""`

csv.DictWriter

```
summary = [{"name": "김민준", "total": 82.2, "grade": "B"}, ...]

with open("out/summary.csv", "w", encoding="utf-8", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=["name", "total", "grade"])
    writer.writeheader()
    writer.writerows(summary)
```

- `newline=""` → Windows에서 빈 줄 끼어듦 방지
- 엑셀에서 한글이 깨지면 `encoding="utf-8-sig"`

try / except / else / finally

```
def to_score(text):
    try:
        value = int(text.strip())      # 오류가 날 수 있는 코드
    except ValueError:
        return None                    # 그 오류가 났을 때
    else:
        return value                   # 오류가 없었을 때
    finally:
        print("변환 시도 끝")          # 항상
```

입력	`"85"`	`" 92 "`	`"abc"`	`""`
결과	85	92	None	None

오류 종류별로 처리

```
try:
    with open(path, encoding="utf-8", newline="") as f:
        values = [int(r[column]) for r in csv.DictReader(f)]
    return sum(values) / len(values)
except FileNotFoundError:
    return "파일 없음"
except KeyError:
    return f"'{column}' 열 없음"
except ValueError as e:
    return f"숫자 변환 실패 ({e})"
except ZeroDivisionError:
    return "데이터 없음"
```

raise: 이유를 담아 알리기

```
def parse_int(text, field, low, high, required=True):
    text = text.strip()
    if text == "":
        if required:
            raise ValueError(f"{field}: 값이 비어 있음")
        return None
    try:
        value = int(text)
    except ValueError:
        raise ValueError(f"{field}: 숫자가 아님 ({text!r})") from None
    if not low <= value <= high:
        raise ValueError(f"{field}: {low}~{high} 범위 밖 ({value})")
    return value
```

실습 1 · 2 (Level 1 → Level 2)

Level 1 따라하기

- 텍스트 파일 쓰기 · 이어 쓰기 · 읽기, `pathlib`
- `DictReader`로 읽고 문자열 함정 확인, `DictWriter`로 저장
- `try/except/else/finally`, 오류 종류별 처리, `raise`

Level 2 바꾸기

- `to_score`에 **0~100 범위 검사** 추가
- `"a"` ↔ `"w"` 모드를 바꿔 결과 비교

실습 3 (Level 3): 검증 규칙

열	규칙
`student_id`	8자리 숫자, **중복 불가**
`name`	비어 있으면 안 됨
`year`	1~4 정수
`midterm`, `final`	필수, 0~100 정수 (공백 허용)
`assignment`	선택(빈 칸 → `None`), 있으면 0~100
`attendance`	0~16 정수

출력: `students_clean.csv` + `error_report.csv` (줄 번호, 학번, 이유)

행 단위 검증 → 분리

```
def process_file(path):
    valid, errors, seen_ids = [], [], set()
    with open(path, encoding="utf-8", newline="") as f:
        for line_no, row in enumerate(csv.DictReader(f), start=2):
            try:
                record = validate_row(row)
                if record["student_id"] in seen_ids:
                    raise ValueError(f"student_id: 중복 ({record['student_id']})")
            except ValueError as e:
                errors.append({"line": line_no, "student_id": row["student_id"],
                               "reason": str(e)})
            else:
                seen_ids.add(record["student_id"])
                valid.append(record)
    return valid, errors
```

오류 보고서

```
line,student_id,reason
5,20261014,name: 값이 비어 있음
6,20261015,midterm: 숫자가 아님 ('abc')
7,20261016,final: 0~100 범위 밖 (120)
9,20261018,midterm: 0~100 범위 밖 (-5)
10,20261012,student_id: 중복 (20261012)
12,20261020,year: 1~4 범위 밖 (5)
16,20261024,final: 값이 비어 있음
17,20261025,attendance: 0~16 범위 밖 (20)
```

줄 번호 (헤더 = 1번 줄) → 원본을 바로 찾아 고칠 수 있다

결과도 검증한다

```
assert len(valid) + len(errors) == total_rows      # 빠진 행 없음

re_valid, re_errors = process_file("out/students_clean.csv")
assert re_errors == []                              # 정상 파일은 다시 검증해도 통과
assert len({r["student_id"] for r in valid}) == len(valid)   # 중복 없음
```

> 처리 결과를 **다시 입력으로 넣어** 확인하는 것은 강력한 검증 방법

AI 코드: except: pass

```
def midterm_average_ai(path):
    total, count = 0, 0
    with open(path, encoding="utf-8", newline="") as f:
        for row in csv.DictReader(f):
            try:
                score = int(row["midterm"])
                if 0 <= score <= 100:
                    total += score
                    conut += 1              # 오타!
            except:
                pass
    return total / count if count else 0
```

결과 **0**, 오류 메시지 **없음** — 오타가 모든 행에서 조용히 삼켜졌다

예외 처리 원칙

1. **예상한 오류만** 잡는다 → `except ValueError:`
2. 건너뛰더라도 **몇 건을 왜** 건너뛰었는지 기록한다
3. 예상하지 못한 오류(오타, 없는 열)는 **그대로 터지게** 둔다

건너뛸 오류	드러나야 할 오류
사용자가 입력한 `"abc"`	변수 이름 오타
빈 칸	없는 열 이름
범위 밖 점수	파일 경로 실수

AI에게 예외 처리를 요청할 때

```
- except: 나 except Exception: pass 는 쓰지 마세요.
- int 변환 실패처럼 예상 가능한 ValueError만 잡고,
  건너뛴 행의 줄 번호와 이유를 함께 반환하세요.
- 파일이 없으면 FileNotFoundError가 그대로 발생하게 두세요.
```

받은 코드에서 확인할 것: `except` 뒤에 **구체적인 오류 이름**이 있는가? 건너뛴 기록이 남는가?

흔한 실수 모음

실수	결과	해결
`"w"` 로 열어 기존 파일 덮어씀	데이터 손실	모드 확인, `"a"`
`encoding` 생략	한글 깨짐 / `UnicodeDecodeError`	`encoding="utf-8"`
CSV 값을 바로 계산	`"78" + "85" == "7885"`	`int()` 변환
`except:` 로 모두 잡기	버그가 숨음	구체적인 예외만
줄 번호를 0부터	원본과 어긋남	`enumerate(..., start=2)`

이번 주 과제 (10점)

문제	내용	배점
1	`safe_int(text, default, low, high)`	2
2	`save_lines` · `append_line` · `count_lines`	2
3	`dept_average(path, column)` CSV 학과별 평균	3
4	`validate_enrollment` 검증 보고서 (AI 협업)	3

문제 4: 규칙 순서, 줄 번호, **정상 통과한 학번만** 중복 기준

자기 점검

1. `"w"`와 `"a"` 모드의 차이는?
2. `DictReader`로 읽은 `"78"`과 `"85"`를 더하면?
3. `try` 블록의 `else`는 언제 실행될까?
4. `except: pass`가 위험한 이유를 예로 설명하면?
5. 오류 보고서에 줄 번호가 필요한 이유는?

정리와 다음 주

오늘	핵심
파일	`with open(..., encoding="utf-8")`, `pathlib`
CSV	`DictReader` / `DictWriter`, `newline=""`, 값은 문자열
예외	예상한 오류만 잡기, `raise`로 이유 전달
견고함	행 단위 검증 → 분리 → 보고서 → 재검증
AI	`except: pass` 금지, 건너뛴 이유 기록

다음 주 · Week 07 클래스와 객체지향 설계 — 도서 대출 관리